

AL2002 Artificial Intelligence Lab Manual
ver 2.0

Saad Ahmad, BS

Inst., Dept. of Computer Science

National University of Computer & Emerging Sciences

Artificial Intelligence Lab Manual

Saad Ahmad

Instructor, Dept. of Computer Science

National University of Computer & Emerging Sciences

160 Industrial Estate, Peshawar, KPK, Pakistan

Version History

1.0 Spring 2026: Compilation of all previous labs in book format with minor updates & corrections {**Saad Ahmad**}

2.0 Spring 2026: Restructured manual: updated chapter titles, added Simulated Annealing, new CSP chapter (Lab 9), Linear & Logistic Regression in Ch. 13, improved heuristics explanation, added TikZ figures & code snippets {**Saad Ahmad**}

This manual is customized for course AL2002 Artificial Intelligence taught at the National University of Computer & Emerging Sciences, Peshawar, Pakistan. The lab portion of this course provides cover for 1 credit hour. Evaluation for this lab is at discretion of your instructor and/or teacher.

The manual can also be used as a learning tool for other artificial intelligence courses both at undergraduate or graduate level. Each chapter in this manual is intended to serve as a separate lab, although instructors should note that some chapter's require more than one lab sessions to complete.

Contents

1	Supervised Learning: Logistic Regression	5
1.1	From Regression to Classification	5
1.2	The Sigmoid Function: The Heart of Logistic Regression	6
1.3	The Logistic Regression Model	6
1.4	The Cost Function: How Wrong Are We?	7
1.5	Learning the Weights: Gradient Descent	8
1.6	Python Implementation: Building Logistic Regression from Scratch	8
1.6.1	Part 0: The Dataset	9
1.6.2	Part 1: Loading the Data with pandas	9
1.6.3	Part 2: Splitting Features and Target	9
1.6.4	Part 3: Feature Scaling (Very Important!)	9
1.6.5	Part 4: Train/Test Split	10
1.6.6	Part 5: The Sigmoid and Hypothesis	11
1.6.7	Part 6: The Cost Function	11
1.6.8	Part 7: Gradient Descent Training Loop	11
1.6.9	Part 8: Making Predictions and Measuring Accuracy	12
1.7	Visualising the Results	12
1.7.1	1. The Loss Curve	12
1.7.2	2. The Confusion Matrix	12
1.7.3	3. The Decision Boundary (using only 2 features)	13
1.8	Example 2: Tumor Classification (Multiple Features)	14
1.9	Example 3: Spam Detection (Numeric Features)	15
1.10	Common Pitfalls	15
1.11	Key Concepts Summary	16
1.12	Lab Tasks	16

Chapter 1

Supervised Learning: Logistic Regression

1.1 From Regression to Classification

In our previous labs we have explored **Naive Bayes** (a probability based classifier) and **K-Means** (an unsupervised grouper). Now we move to one of the most widely used and beautifully simple classification algorithms in the world: **Logistic Regression**.

The name is a little misleading. Although the word “Regression” appears in the title, Logistic Regression is actually a **classification algorithm**. It is used to answer *Yes/No* questions, such as:

- Will this student **pass** the exam? (Yes / No)
- Is this email **spam**? (Yes / No)
- Is this tumor **malignant** or **benign**? (1 / 0)
- Will this customer **click** the advertisement? (1 / 0)

Imagine you tried to use a straight line (Linear Regression) to predict whether a student passes (1) or fails (0). The line would happily produce values like -0.4 , 1.7 , 3.2 and so on. But “passing the exam” is not a number, it is a *decision*. We need an output that lives politely between 0 and 1 and can be read as a **probability**. That is exactly what Logistic Regression gives us.

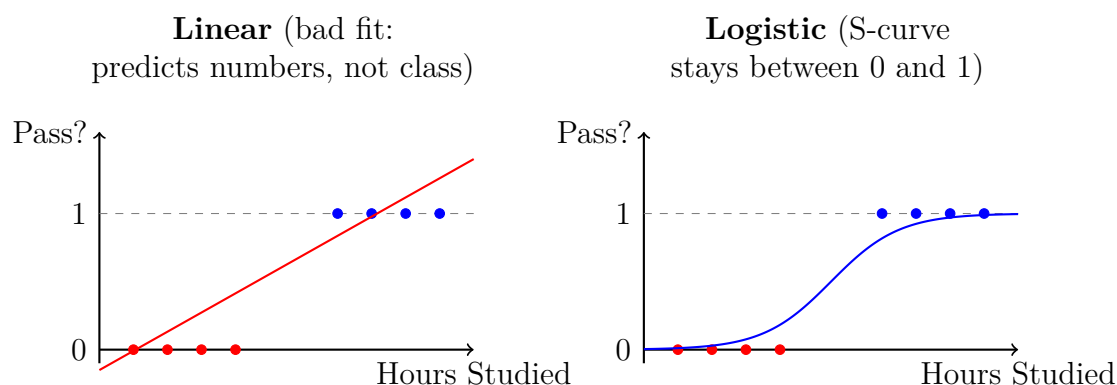


Figure 1.1: Why Linear Regression is a bad fit for Yes/No data, and how the Logistic (S-shaped) curve handles it gracefully.

1.2 The Sigmoid Function: The Heart of Logistic Regression

To squash any number into the range $[0, 1]$ we use a special function called the **Sigmoid** (or *logistic*) function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

No matter what number you feed into σ , the output is always between 0 and 1. Here are some intuitions:

- When z is a **large positive number** (say $z = 5$), $\sigma(z) \approx 0.99$ (very close to 1).
- When $z = 0$, $\sigma(z) = 0.5$ exactly. This is the “decision boundary”.
- When z is a **large negative number** (say $z = -5$), $\sigma(z) \approx 0.01$ (very close to 0).

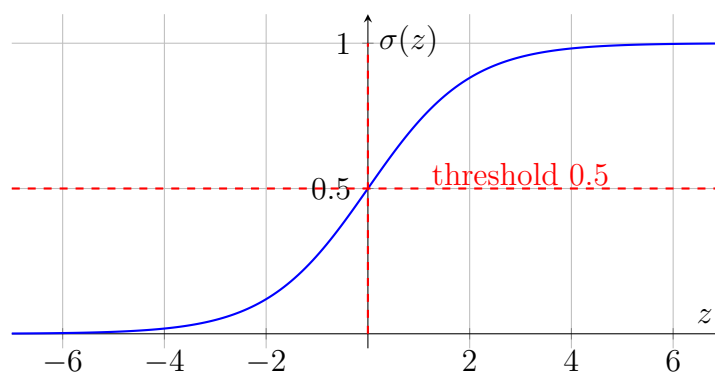


Figure 1.2: The Sigmoid function $\sigma(z)$ smoothly maps any real number z into a probability between 0 and 1.

In code, the sigmoid is just one line:

```
1 import numpy as np
2
3 def sigmoid(z):
4     return 1 / (1 + np.exp(-z))
```

Checkpoint 1: Verify the Sigmoid by Hand

Compute (using a calculator) the value of $\sigma(z)$ for $z = 0$, $z = 2$ and $z = -2$. What do you observe about $\sigma(2)$ and $\sigma(-2)$? Why is it useful that the curve is *symmetric* around $z = 0$?

1.3 The Logistic Regression Model

Logistic Regression is essentially a two-step computation:

1. **Compute a linear combination of inputs** (this part looks like Linear Regression):

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

2. Pass that number through the **sigmoid** to get a probability:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

We then pick a threshold (commonly 0.5):

$$\text{prediction} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{if } \hat{y} < 0.5 \end{cases}$$

The numbers $w_1, w_2, \dots, w_n$ are called **weights** (one for each input feature) and b is the **bias** (sometimes written as w_0). Training the model means finding the values of $\mathbf{w}$ and b that best separate the two classes.

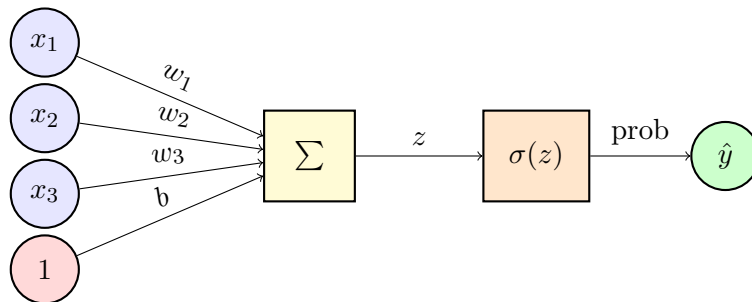


Figure 1.3: Architecture of Logistic Regression: linear combination $\rightarrow$ sigmoid $\rightarrow$ probability.

1.4 The Cost Function: How Wrong Are We?

To train the model, we need a way to measure how bad our current w and b are. In Linear Regression we used **Mean Squared Error (MSE)**. For Logistic Regression we use a different, more appropriate cost called **Binary Cross-Entropy** (also known as the *Log Loss*):

$$J(w, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

Don't be scared by the formula! In plain English:

- If the true label $y_i = 1$ and the model predicts $\hat{y}_i$ close to 1, then $\log(\hat{y}_i) \approx 0$, so the cost is tiny. *Good!*
- If the true label $y_i = 1$ but the model predicts $\hat{y}_i$ close to 0, then $\log(\hat{y}_i) \approx -\infty$, so the cost is huge. *Punish the model!*
- The same logic applies (flipped) when $y_i = 0$.

The intuition is: “be brave and confident only when you are right; be heavily penalized when you are confidently wrong.”

Why not MSE for classification? Because the sigmoid combined with squared error creates a *non-convex* loss surface that has lots of flat valleys, where Gradient Descent gets stuck. Cross-Entropy with sigmoid is mathematically guaranteed to be convex (a single bowl-shaped minimum), so optimization is much smoother.

1.5 Learning the Weights: Gradient Descent

Gradient Descent is a fancy way of saying “walk downhill until you hit the bottom of the bowl.” It is the same idea you saw with the Perceptron in earlier labs: tweak the weights a tiny bit at a time so the cost decreases.

The update rules for Logistic Regression have a beautifully clean form:

$$w_j := w_j - \alpha \cdot \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

$$b := b - \alpha \cdot \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Where:

- α is the **learning rate** (a small positive number like 0.01).
- $(\hat{y}_i - y_i)$ is the error for sample i .
- x_{ij} is the value of feature j for sample i .

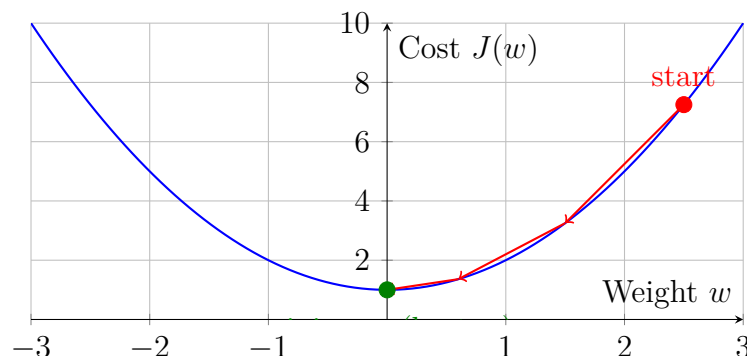


Figure 1.4: Gradient Descent: keep stepping in the direction that decreases the cost until you reach the bottom.

Checkpoint 2: Reasoning about the Learning Rate

What do you think happens if α is set to a very large value like 5.0? What if it is too small like 0.00001? Sketch the path Gradient Descent would take in each case.

1.6 Python Implementation: Building Logistic Regression from Scratch

We will now build the entire Logistic Regression algorithm using only `numpy`, `pandas`, and `matplotlib`. We will load a real (small) CSV dataset, train our own model, and even visualise the decision boundary and confusion matrix **without** using `scikit-learn`. This will give you a deep understanding of what really happens inside the library.

1.6.1 Part 0: The Dataset

We will use the dataset `students_pass_fail.csv`. Each row describes a student with three features and a target column:

- **StudyHours**: average hours of study per day.
- **Attendance**: class attendance percentage.
- **PrevGrade**: percentage in the previous course.
- **Pass**: target. 1 if the student passed, 0 if they failed.

1.6.2 Part 1: Loading the Data with pandas

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 df = pd.read_csv("students_pass_fail.csv")
6 print(df.head())
7 print("Shape:", df.shape)
8 print("Class balance:\n", df["Pass"].value_counts())

```

What is happening? `pd.read_csv` reads our CSV into a DataFrame (a fancy spreadsheet). `df.head()` shows the first 5 rows so we can quickly eyeball the data. `value_counts()` on the target column tells us how many passing vs failing students we have, which is important to check before training.

1.6.3 Part 2: Splitting Features and Target

```

1 X = df[["StudyHours", "Attendance", "PrevGrade"]].values # shape: (n, 3)
2 y = df["Pass"].values # shape: (n,)
3
4 print("X shape:", X.shape)
5 print("y shape:", y.shape)

```

Why .values? It converts the DataFrame columns into plain NumPy arrays which are much faster for math operations.

1.6.4 Part 3: Feature Scaling (Very Important!)

The three features are on very different scales: **StudyHours** is between 1 and 9, **Attendance** is in 0–100, and **PrevGrade** is also in 0–100. If we feed raw values into Gradient Descent, the larger features will dominate the weight updates and the algorithm will struggle. We fix this with **standardisation** (subtract mean, divide by standard deviation):

```

1 def standardize(X):
2     mean = X.mean(axis=0)
3     std = X.std(axis=0)
4     return (X - mean) / std, mean, std
5
6 X_scaled, mean, std = standardize(X)
7 print("After scaling, mean per column:", X_scaled.mean(axis=0))
8 print("After scaling, std per column:", X_scaled.std(axis=0))

```

What is happening? After standardisation, every feature has a mean of about 0 and a standard deviation of 1. We *remember* the `mean` and `std` so we can apply the exact same transformation to new test data later.

1.6.5 Part 4: Train/Test Split

```

1 def train_test_split_manual(X, y, test_ratio=0.2, seed=42):
2     np.random.seed(seed)
3     n = len(X)
4     indices = np.random.permutation(n)
5     test_size = int(n * test_ratio)
6     test_idx, train_idx = indices[:test_size], indices[test_size:]
7     return X[train_idx], X[test_idx], y[train_idx], y[test_idx]
8
9 X_train, X_test, y_train, y_test = train_test_split_manual(X_scaled, y)
10 print("Train:", X_train.shape, "Test:", X_test.shape)

```

Detailed Explanation (line-by-line):

- `def train_test_split_manual(...)` defines our own split function so students understand what libraries do behind the scenes.
- `np.random.seed(seed)` fixes randomness. With the same seed (e.g., 42), you will always get the same train/test split, which is useful for reproducible experiments.
- `n = len(X)` stores the number of samples (rows) in the feature matrix.
- `indices = np.random.permutation(n)` creates a randomly shuffled list of indices from 0 to `n-1`. This avoids bias from original row order in the CSV file.
- `test_size = int(n * test_ratio)` computes how many samples should go to test data. If `n = 100` and `test_ratio = 0.2`, then test size becomes 20.
- `test_idx, train_idx = indices[:test_size], indices[test_size:]` slices the shuffled indices:
 - first chunk → test set indices
 - remaining chunk → training set indices
- `return X[train_idx], X[test_idx], y[train_idx], y[test_idx]` returns four arrays in a clean order: `X_train`, `X_test`, `y_train`, `y_test`.

Why do we split at all? If we train and test on the same data, the model can “memorize” patterns and appear perfect even when it cannot generalize. A test set simulates unseen real-world data and gives an honest estimate of performance.

Reading the output shapes: `print("Train:", X_train.shape, "Test:", X_test.shape)` tells us how many rows ended up in each set. For example, with 45 samples and `test_ratio=0.2`, you should see roughly 36 training rows and 9 test rows.

Checkpoint: Change `test_ratio` from 0.2 to 0.3 and 0.4. Observe how training size and test size change, and discuss whether accuracy becomes more stable or more noisy.

1.6.6 Part 5: The Sigmoid and Hypothesis

```

1 def sigmoid(z):
2     return 1 / (1 + np.exp(-z))
3
4 def predict_proba(X, w, b):
5     z = X @ w + b          # @ is matrix multiplication
6     return sigmoid(z)

```

What is happening? $X @ w$ computes the dot product for every row: a vectorised way of saying “ $w_1x_1 + w_2x_2 + w_3x_3$ for every student”. NumPy makes this lightning fast.

1.6.7 Part 6: The Cost Function

```

1 def compute_cost(X, y, w, b):
2     n = len(y)
3     y_hat = predict_proba(X, w, b)
4     # We add a tiny epsilon to avoid log(0) which is -infinity
5     eps = 1e-9
6     cost = -np.mean(y * np.log(y_hat + eps) + (1 - y) * np.log(1 - y_hat + eps))
7     return cost

```

What is happening? We turn the math formula straight into NumPy code. The `eps` trick saves us from blowing up if a prediction is exactly 0 or 1.

1.6.8 Part 7: Gradient Descent Training Loop

```

1 def train_logistic_regression(X, y, lr=0.1, epochs=2000):
2     n, num_features = X.shape
3     w = np.zeros(num_features)
4     b = 0.0
5     history = []    # store cost at every epoch for plotting
6
7     for epoch in range(epochs):
8         y_hat = predict_proba(X, w, b)
9         error = y_hat - y          # shape (n,)
10
11         # Gradients
12         dw = (X.T @ error) / n    # shape (num_features,)
13         db = np.mean(error)
14
15         # Update parameters
16         w = w - lr * dw
17         b = b - lr * db
18
19         cost = compute_cost(X, y, w, b)
20         history.append(cost)
21
22         if epoch % 200 == 0:
23             print(f"Epoch {epoch:4d} | Cost = {cost:.4f}")
24
25     return w, b, history
26
27 w, b, history = train_logistic_regression(X_train, y_train, lr=0.1, epochs=2000)
28 print("\nFinal weights:", w)
29 print("Final bias:", b)

```

What is happening? We start with all zero weights, then loop `epochs` times. In every iteration we (1) make predictions, (2) compute the error, (3) compute gradients of the cost with respect to w and b , and (4) nudge the parameters in the direction that lowers the cost. Notice how each gradient is just an average over the dataset. This is a complete **batch** Gradient Descent.

1.6.9 Part 8: Making Predictions and Measuring Accuracy

```

1 def predict(X, w, b, threshold=0.5):
2     probs = predict_proba(X, w, b)
3     return (probs >= threshold).astype(int)
4
5 def accuracy(y_true, y_pred):
6     return np.mean(y_true == y_pred)
7
8 y_pred_train = predict(X_train, w, b)
9 y_pred_test  = predict(X_test,  w, b)
10
11 print("Train accuracy:", accuracy(y_train, y_pred_train))
12 print("Test  accuracy:", accuracy(y_test,  y_pred_test))

```

What is happening? If the predicted probability is ≥ 0.5 we classify as “Pass”, otherwise “Fail”. Accuracy is just the percentage of predictions that matched the truth.

1.7 Visualising the Results

A model is much easier to trust when you can *see* how it learned. We will produce three useful plots.

1.7.1 1. The Loss Curve

```

1 plt.figure(figsize=(7, 4))
2 plt.plot(history, color="steelblue")
3 plt.title("Training Cost over Epochs")
4 plt.xlabel("Epoch")
5 plt.ylabel("Binary Cross-Entropy Cost")
6 plt.grid(True, alpha=0.3)
7 plt.show()

```

A healthy training curve goes *down* smoothly. If it goes up, the learning rate is too large; if it is flat from the start, it is too small.

1.7.2 2. The Confusion Matrix

A confusion matrix tells us *what kind of mistakes* the model makes. It is a 2×2 table:

	Predicted: 0	Predicted: 1
Actual: 0	True Negative (TN)	False Positive (FP)
Actual: 1	False Negative (FN)	True Positive (TP)

Table 1.1: The Confusion Matrix layout.

```

1 def confusion_matrix_manual(y_true, y_pred):
2     tp = np.sum((y_true == 1) & (y_pred == 1))
3     tn = np.sum((y_true == 0) & (y_pred == 0))
4     fp = np.sum((y_true == 0) & (y_pred == 1))
5     fn = np.sum((y_true == 1) & (y_pred == 0))
6     return np.array([[tn, fp],
7                      [fn, tp]])
8
9 cm = confusion_matrix_manual(y_test, y_pred_test)
10 print("Confusion Matrix:\n", cm)

```

We can visualise the matrix with a pretty heatmap. `seaborn` is the cleanest way; if you do not have it installed, `matplotlib` alone works just as well.

```

1 import seaborn as sns # optional; pip install seaborn
2
3 plt.figure(figsize=(5, 4))
4 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
5             xticklabels=["Predicted Fail", "Predicted Pass"],
6             yticklabels=["Actual Fail", "Actual Pass"])
7 plt.title("Confusion Matrix")
8 plt.show()

```

Pure-matplotlib alternative (no seaborn):

```

1 fig, ax = plt.subplots(figsize=(5, 4))
2 im = ax.imshow(cm, cmap="Blues")
3 ax.set_xticks([0, 1]); ax.set_yticks([0, 1])
4 ax.set_xticklabels(["Pred Fail", "Pred Pass"])
5 ax.set_yticklabels(["Act Fail", "Act Pass"])
6 for i in range(2):
7     for j in range(2):
8         ax.text(j, i, cm[i, j], ha="center", va="center",
9               color="white" if cm[i, j] > cm.max()/2 else "black")
10 plt.colorbar(im); plt.title("Confusion Matrix"); plt.show()

```

From the confusion matrix we can compute three important metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

```

1 def metrics_from_cm(cm):
2     tn, fp = cm[0]
3     fn, tp = cm[1]
4     accuracy = (tp + tn) / cm.sum()
5     precision = tp / (tp + fp) if (tp + fp) else 0
6     recall = tp / (tp + fn) if (tp + fn) else 0
7     f1 = 2 * precision * recall / (precision + recall) if (precision +
8         recall) else 0
9     return accuracy, precision, recall, f1
10
11 acc, prec, rec, f1 = metrics_from_cm(cm)
12 print(f"Accuracy : {acc:.3f}")
13 print(f"Precision: {prec:.3f}")
14 print(f"Recall : {rec:.3f}")
15 print(f"F1-Score : {f1:.3f}")

```

1.7.3 3. The Decision Boundary (using only 2 features)

To actually *see* the line our model learned, let's train a separate small model using only two features (`StudyHours` and `PrevGrade`) and plot the boundary.

```

1 X2 = df[["StudyHours", "PrevGrade"]].values
2 X2_scaled, m2, s2 = standardize(X2)
3 w2, b2, _ = train_logistic_regression(X2_scaled, y, lr=0.1, epochs=2000)
4
5 # Make a fine grid covering the feature space
6 xx, yy = np.meshgrid(np.linspace(X2_scaled[:, 0].min()-1, X2_scaled[:, 0].max()
7                        +1, 200),
8                      np.linspace(X2_scaled[:, 1].min()-1, X2_scaled[:, 1].max()
9                        +1, 200))
10
11 grid = np.c_[xx.ravel(), yy.ravel()]
12 probs = predict_proba(grid, w2, b2).reshape(xx.shape)
13
14 plt.figure(figsize=(7, 5))
15 plt.contourf(xx, yy, probs, levels=20, cmap="RdBu", alpha=0.7)
16 plt.contour(xx, yy, probs, levels=[0.5], colors="black", linewidths=2)
17 plt.scatter(X2_scaled[y == 0, 0], X2_scaled[y == 0, 1], color="red",
18             edgecolor="k", label="Fail")
19 plt.scatter(X2_scaled[y == 1, 0], X2_scaled[y == 1, 1], color="blue",
20             edgecolor="k", label="Pass")
21 plt.xlabel("StudyHours (scaled)"); plt.ylabel("PrevGrade (scaled)")
22 plt.title("Decision Boundary of Logistic Regression")
23 plt.legend(); plt.show()

```

The black line is exactly where $\sigma(z) = 0.5$, i.e. the boundary between the two predicted classes. The colored background shows how confident the model is in each region.

Student Activity 1: Run the full pipeline and answer:

1. What was your final test accuracy?
2. Look at the confusion matrix: were there any *false positives* (predicted Pass but actually Fail)? Which is worse for a student in real life: a false positive or a false negative? Justify your answer.
3. Change the learning rate to 0.001. How does the loss curve change?

1.8 Example 2: Tumor Classification (Multiple Features)

Logistic Regression really shines when working with multiple features. The same code we wrote above works on *any* dataset, regardless of how many features it has. Let us try a more medically inspired example using `tumor_classification.csv`.

```

1 df_tumor = pd.read_csv("tumor_classification.csv")
2 print(df_tumor.head())
3
4 X_t = df_tumor[["MeanRadius", "MeanTexture"]].values
5 y_t = df_tumor["Malignant"].values
6 X_t_scaled, mt, st = standardize(X_t)
7
8 X_tr, X_te, y_tr, y_te = train_test_split_manual(X_t_scaled, y_t, test_ratio
9                                                =0.25)
10 w_t, b_t, hist_t = train_logistic_regression(X_tr, y_tr, lr=0.1, epochs=2000)
11
12 y_pred_t = predict(X_te, w_t, b_t)
13 cm_t = confusion_matrix_manual(y_te, y_pred_t)

```

```

13 acc_t, prec_t, rec_t, f1_t = metrics_from_cm(cm_t)
14 print(f"Tumor model -> Acc {acc_t:.2f}, Prec {prec_t:.2f}, Rec {rec_t:.2f}, F1
    {f1_t:.2f}")

```

Reflection: In medicine, **Recall** is usually more important than precision because missing a malignant tumor (false negative) is far worse than a false alarm. This is why doctors often *lower* the decision threshold from 0.5 to maybe 0.3, even at the cost of more false positives. Try it!

Student Activity 2: Re-run prediction on the tumor test set with thresholds of 0.3, 0.5, and 0.7. Print accuracy, precision and recall for each threshold. Which threshold would you choose if you were the medical director?

1.9 Example 3: Spam Detection (Numeric Features)

The third dataset, `email_spam.csv`, contains numeric features extracted from emails such as how many times the word *free* occurs, how many times *money* occurs, the number of links, and the ratio of capital letters. The target is whether the email is spam.

```

1 df_spam = pd.read_csv("email_spam.csv")
2 X_s = df_spam[["FreeCount", "MoneyCount", "LinkCount", "CapsRatio"]].values
3 y_s = df_spam["Spam"].values
4
5 X_s_scaled, _, _ = standardize(X_s)
6 X_str, X_ste, y_str, y_ste = train_test_split_manual(X_s_scaled, y_s)
7
8 w_s, b_s, _ = train_logistic_regression(X_str, y_str, lr=0.1, epochs=3000)
9 print("Test accuracy:", accuracy(y_ste, predict(X_ste, w_s, b_s)))
10 print("Learned weights (Free, Money, Link, Caps):", w_s)

```

Interpretation tip: The learned weights tell you which feature *pushes* the prediction towards spam. The largest positive weight wins. If the weight for **FreeCount** is the largest, then the word *free* appearing in an email is the strongest spam signal in this small dataset.

1.10 Common Pitfalls

- **Forgetting to scale features.** If one feature is in the range 0–1 and another is 0–100000, Gradient Descent will be miserable. Always standardise first.
- **Learning rate too high.** The cost will go UP and then explode (NaNs everywhere). Reduce `lr` by 10×.
- **Learning rate too low.** The cost barely moves. Try increasing `lr` or running more epochs.
- **Imbalanced classes.** If 95% of your data is class 0, the model can “cheat” and predict 0 always for 95% accuracy. Always check precision/recall, not just accuracy!
- **log(0) explosions.** Always add a small `eps` when taking the log of probabilities.

1.11 Key Concepts Summary

- **Logistic Regression** is a linear classifier that outputs probabilities.
- The **Sigmoid** function maps any real number into $[0, 1]$.
- The **Hypothesis** is $\hat{y} = \sigma(w \cdot x + b)$.
- The **Decision Boundary** is the line/plane where $\sigma(z) = 0.5$, i.e. where $z = 0$.
- The **Cost Function** is Binary Cross-Entropy (Log Loss).
- **Gradient Descent** updates weights using $w := w - \alpha \cdot \nabla J(w)$.
- Always evaluate using a **Confusion Matrix** and metrics like *Precision*, *Recall*, and *F1*, not just *Accuracy*.

1.12 Lab Tasks

Task 1: Implement and Train

Using `students_pass_fail.csv`, complete the entire pipeline shown in this chapter. Print:

- Final learned w and b .
- Train and Test accuracy.
- The full Confusion Matrix.
- The training loss curve plot.

Task 2: Effect of Learning Rate

Train the model with $\text{lr} = 0.001, 0.01, 0.1$, and 1.0 . On a single `matplotlib` figure, plot all four loss curves with different colors and a legend. Discuss which learning rate is the most stable.

Task 3: Effect of Threshold

For the tumor dataset, sweep the decision threshold from 0.1 to 0.9 in steps of 0.1 . For each threshold, record *precision* and *recall* and plot them on the same figure. This is your first taste of a **Precision–Recall Curve**.

```

1 thresholds = np.arange(0.1, 1.0, 0.1)
2 precisions, recalls = [], []
3 for t in thresholds:
4     y_p = predict(X_te, w_t, b_t, threshold=t)
5     cm = confusion_matrix_manual(y_te, y_p)
6     _, p, r, _ = metrics_from_cm(cm)
7     precisions.append(p); recalls.append(r)
8
9 plt.plot(thresholds, precisions, "o-", label="Precision")
10 plt.plot(thresholds, recalls, "s-", label="Recall")
11 plt.xlabel("Threshold"); plt.legend(); plt.grid(True); plt.show()

```


Task 4: Predict on a New Student

Imagine a new student arrives:

StudyHours = 5, Attendance = 80, PrevGrade = 70

Use your trained model to predict whether this student will pass. Remember to apply the *same* standardisation (using the training `mean` and `std`) before calling `predict`.

Task 5: Challenge - Multi-Class Hint

Logistic Regression as we built it only handles two classes. For three or more classes, the standard trick is called **One-vs-Rest**: train one binary Logistic Regression model per class (“is it class 0 or not?”, “is it class 1 or not?”, ...) and at prediction time pick the class with the highest probability. Sketch (in pseudocode) how you would extend our code to support three classes on a synthetic three-class dataset.

Bibliography

- [1] Agents in artificial intelligence.
- [2] Artificial intelligence.
- [3] Artificial intelligence - agents and environments.
- [4] Cs6-1 lab 06.
- [5] Matplotlib documentation.
- [6] Networkx - python software package for complex networks.
- [7] Numpy documentation.
- [8] pandas documentation.
- [9] seaborn documentation.
- [10] Types of artificial intelligence.
- [11] What is intelligent agent in ai - types and function.
- [12] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [13] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [14] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition, 2009.
- [15] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.
- [16] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
- [17] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [18] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.